

Chapter 6. Monitoring ML Applications

According to The Institute for Ethical AI & Machine Learning, 42% of ML projects do not have monitoring in place (see [Figure 6-1](#)). Even though that figure is better than the one from the year before (50% with no monitoring), it shows how immature most companies are when it comes to MLOps.

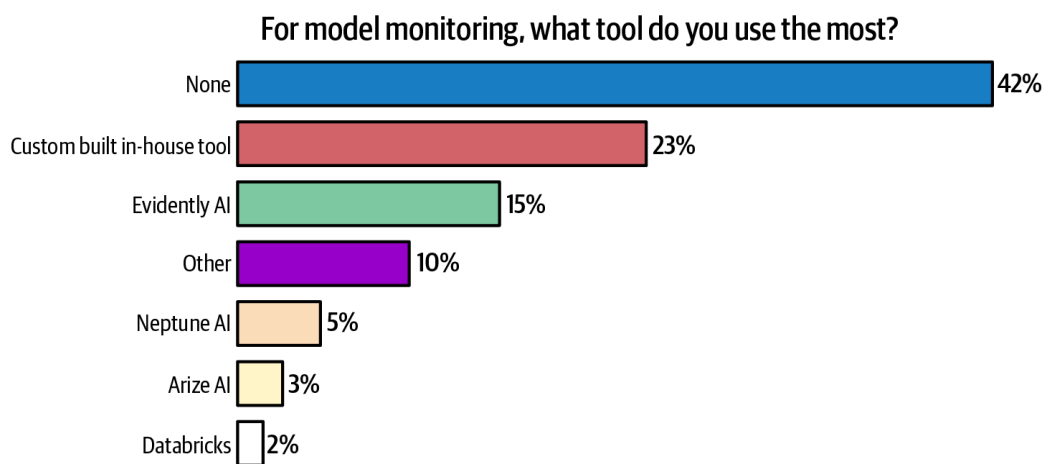


Figure 6-1. Updated image from the [State of Production ML in 2025 survey](#).

But monitoring isn't a "nice-to-have." Without it, models fail silently. Predictions degrade. Bias can increase over time. Infrastructure costs can spiral. And perhaps most damaging of all, stakeholders will lose trust in the system when performance suddenly drops and no one knows why. In regulated industries, lack of monitoring can even create legal and compliance risk.

By the end of this chapter, you will understand all of the following:

- The different layers of ML monitoring
- How to design a practical monitoring strategy
- What metrics actually matter in production
- How to set alerts on aspects that matter
- How monitoring connects to retraining and continuous improvement

In short, this chapter will move you from “Model deployed” to “Model managed.”

What to Monitor

Monitoring requirements for ML projects are highly use-case dependent. While generic software metrics are important, ML-specific monitoring focuses on the model’s relationship with changing data and its ultimate impact on business outcomes. Let’s walk through a few examples.

Recommender Systems

In a *recommender system*, such as one for an online grocery store, a user receives product suggestions based on their basket content and historical context. The feedback loop in this scenario is rather short; you can quickly determine if a recommended item was added to the basket and purchased.

You can use metrics like precision@K, recall@K, and F-score@K and monitor them over time:

Precision@K

This measures what share of items within the top K positions are relevant.

Recall@K

This measures the share of relevant items captured within the top K positions.

F-score@K

This provides a balanced measure of Precision and Recall at K .

If you want to explore the nuances further, Evidently AI has an [extensive explanation of these metrics](#). Overall, the higher the value of these metrics, the better the quality of the recommender.

However, we should not forget the purpose of the recommender system in the first place—generating monetary value. For the recommender to fulfill its purpose, users must have a *higher basket value* with the recommender than they would with random suggestions or no system at all. If

the basket value isn't increasing, then high performance metrics are misleading. Keep in mind, however, that it often requires a longer feedback loop to gather enough data to confirm this financial impact.

In the case of such a recommender system, we can evaluate model quality offline during pre-deployment, monitor those same performance metrics in near-real time once the system is live, and track the business metrics daily.

Regression and Demand Forecasting

In regression tasks, such as predicting weekly sales by product category, calculating metrics like RMSE in real time is impossible because the ground truth (actual sales numbers) takes a full week to materialize.

However, there are two things we can monitor with a short feedback loop:

Business user feedback

We can monitor for an unusual volume of manual corrections to model outputs.

Estimated model performance

We can utilize methods like direct loss estimation (DLE) to approximate performance metrics before the ground truth is available.

The DLE method works by training a secondary *nanny model* to predict the absolute error of the primary *child model*. While both models use the same input features, the nanny model uses the absolute error of the monitored model as its target. This allows for performance estimation on new observations where the ground truth is not known. For a deeper technical breakdown, check out the [NannyML documentation](#), which offers extensive resources on this approach.

The ultimate performance metric is again the business value. Evaluation should focus on the specific costs associated with over-prediction versus under-prediction and whether the model provides a measurable lift over simple baselines, such as a moving average.

Large Language Models

Monitoring LLMs is even harder because outputs are stochastic and context dependent. In their article “Unsolved Problems in MLOps,” Niall Murphy and Todd Underwood provide an example of how Azure monitors LLM performance:

*Azure uses two ways of validating that new models in the Azure UI are working well: first, a set of LLMs judging the output of the new LLMs, and second, whether or not enough Microsoft employees have hit the equivalent of the “thumbs up” button when exposed to its recommendations.*¹

Monitoring AI projects often comes down to monitoring custom business metrics that are tailored to a specific organization. This need for specialized key performance indicators (KPIs) is a primary reason why many companies develop bespoke monitoring tools.

Regardless of the use case, the goal is to detect *abrupt or gradual changes in model outputs*, which can signal that the world around the model has shifted. This is known as *concept drift*.

For example, imagine a model that predicts housing prices based on features like the age and size of the house, the neighborhood, the year when it was most recently renovated, and the number of bedrooms. Over time, the relationships among these variables and the final price may change. If the government introduces a new housing subsidy for families or if market preferences shift, a large home might suddenly sell for less than historically expected. At that point, because the underlying relationship between house size and price has changed, the model’s original assumptions are no longer valid. This shift, where the logic of the data itself evolves, is the essence of concept drift.

Concept Drift versus Data Drift

In contrast, *data drift* happens when the input data distribution shifts even though the underlying relationships remain the same. Continuing with the housing example, imagine that there’s a spike in new construction in a specific neighborhood, but without any changes to government policy or buyer preferences. The model might begin to underperform simply because it hasn’t encountered enough examples before. In this

scenario, nothing fundamental has changed about how house features relate to price; only the composition of the data has shifted.

Significant data drift, even in features with high predictive power, may not hurt model quality, especially if the model generalizes well. For this reason, drift detection should never be viewed in isolation. It must be combined with model performance estimation and business metrics to determine whether the observed change truly matters.²

Fairness, Bias, and Regulatory Compliance

Another aspect of monitoring is *fairness and bias*, which has become a formal regulatory requirement under the EU Artificial Intelligence Act. Credit-scoring systems, employment systems, and other “high-risk” AI systems must demonstrate fairness, transparency, and risk management throughout their lifecycle, and organizations running such systems must engage in continuous monitoring after deploying them. In practice, this requires organizations to verify that their models perform equitably across different groups. For instance, in a loan approval scenario, applicants with similar financial profiles must receive comparable creditworthiness scores, regardless of demographic attributes like gender or ethnicity. These requirements are discussed in [Chapter 10](#).

System and Infrastructure Health

Besides model- and business-specific aspects, it’s also important to keep an eye on the *generic monitoring* that every software system needs. Even the best model is useless if the system around it is failing.

This includes tracking *system health*, *errors*, *latency*, and *infrastructure costs*. A data pipeline outage or slow feature store can degrade predictions more than any modeling issue. Spikes in serving latency can silently harm user experience, while unnoticed infrastructure scaling can quickly inflate cloud bills. In mature MLOps setups, these signals are integrated into the same observability layer as model and data metrics.

Monitoring, in the end, is about *closing feedback loops* across models, data, and infrastructure so that issues are detected early and can be acted on before they affect users or business outcomes.

To recap, a comprehensive monitoring strategy for ML applications covers four key pillars, each of which has multiple components:

Model performance metrics

Prediction quality

You perform real-time tracking of metrics like precision@K, recall@K, and F-score@K for recommenders, or RMSE, MAE, and MAPE for regression tasks.

Estimated performance

When ground truth arrives late (e.g., weekly or monthly), you use proxy estimates (when possible) to predict model error in near-real time.

Drift and stability

You watch for abrupt or gradual changes in model outputs or input distributions that may indicate concept drift or data drift.

Fairness and bias

You monitor whether the model performs similarly on data associated with different demographic groups (if such data is available).

Business metrics

Value alignment

You connect model output to revenue lift, conversion rates, or reduced waste.

Impact over accuracy

Even if your offline metrics look good, you must make sure your model is moving key business indicators in the right direction. If not, it's not a successful model.

Operational and human-in-the-loop signals

User feedback

You track user interactions and feedback (clicks, purchases, and thumbs up/down).

Manual overrides

You monitor business user overrides or corrections since a rise in manual edits often indicates declining trust or model relevance.

System and infrastructure health

Operational robustness

Monitoring system health, errors, latency, and infrastructure costs is part of every robust MLOps setup.

Cost management

A broken pipeline, a slow feature store, or escalating compute cost can degrade business outcomes as much as a poor model.

Next, we'll take a deep dive into Databricks Data Quality Monitoring to explore how Databricks simplifies these tasks by unifying data and AI observability in a single platform.

Introduction to Databricks Data Quality Monitoring

Databricks Data Quality Monitoring provides built-in capabilities to monitor both data and models in production. It enables you to track the statistical properties and quality of data across selected tables, so it helps you identify issues such as missing values or distribution shifts.

NOTE

You can monitor the input tables that are used to train ML models, as well as serving endpoints. You can do this by first processing inference tables that store model inputs and predictions and then setting up monitoring of the resulting aggregated tables.

Data Quality Monitoring supports two primary automated monitoring scenarios—time series and inference—as well as snapshot mode for custom analysis:

Time series scenario

This is suitable for input tables that are used for model training or feature generation, in which data arrives periodically and includes a timestamp column.

Inference scenario

This is designed for tables that store model inputs and predictions, which allow you to monitor both data quality and model performance over time. Note that you need the ground truth to compute the model performance metrics.

When monitoring is enabled on a table, Databricks automatically creates three core assets:

- A profile metrics table
- A drift metrics table
- A Databricks SQL dashboard for visualization

[Figure 6-2](#) depicts the relationships among these assets.

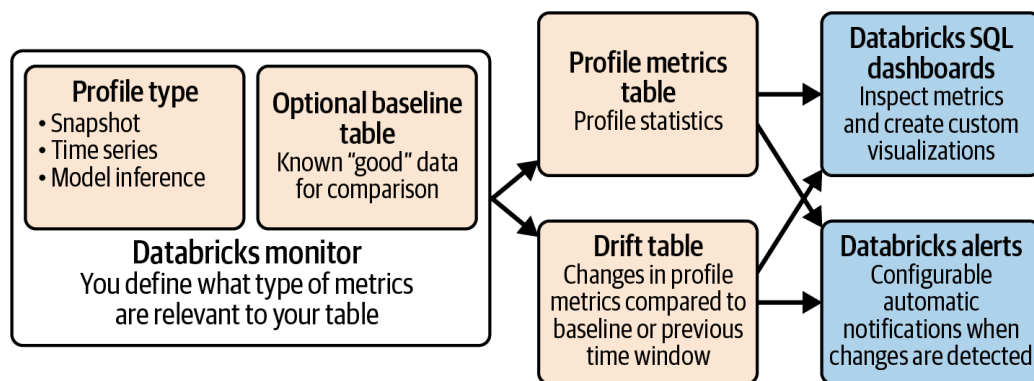


Figure 6-2. The three core monitoring assets, adapted from [Databricks documentation](#)

The *profile metrics table* contains summary statistics for each column and for each combination of the *time window*, *slice*, and *grouping columns*. These include measures such as counts, means, standard deviations, and value distributions. For inference analyses, the table also includes *model accuracy metrics* like precision, recall, or RMSE, depending on the model type.

The *drift metrics table* tracks *changes in data distributions* over time. Rather than focusing on specific values, it measures how much a given metric changes compared to either the *previous time window* (via consecutive drift) or an optional *baseline table*, such as the training dataset, which is used as a fixed reference for comparison. Note that the schema of the baseline table must match the schema of the input table.

Consecutive drift is computed only when a preceding time window exists after aggregation according to the chosen granularity. For time series and inference scenarios, the following time windows are supported:

- 5 minutes
- 30 minutes
- 1 hour
- 1 day
- *N* weeks
- 1 month
- 1 year

If you need more control—for example, to monitor tables without a timestamp column or to analyze data on an ad hoc schedule—you can use a snapshot profile instead. *Snapshot profiles* process the entire table at each run and can optionally compare results to a baseline table. Even without a baseline, Databricks automatically compares metrics between snapshots, to help you detect changes or drift over time.

Across all monitoring modes, you can define *slices* to break down metrics by dimensions such as region, product category, or customer segment. This allows you to identify data quality issues or model drifts that appear only within specific subgroups, even when overall metrics seem stable.

Profile Metrics Table

Let's take a closer look at the profile metrics table structure to understand how we can use it for monitoring purposes.

The table organizes data using several key *grouping columns*:

Window and granularity

This column defines the specific time interval (e.g., a one-hour window) using a `struct<start: timestamp, end: timestamp>`

format.

Model ID (`model_id_col`)

This column is specific to the inference scenario. If an input table contains predictions from multiple model versions (such as during A/B testing), this column allows Data Quality Monitoring to compute separate metrics for each version, rather than aggregating them.

Log type (`log_type`)

This column indicates which table was used to calculate the metrics. When set to `BASELINE`, metrics are computed for the baseline (reference) table; when set to `INPUT`, metrics are computed for the currently monitored table.

Slicing (`slice_key` and `slice_value`)

These columns capture the slicing expression and its value, thus enabling you to compare performance across different categories or segments.

Column metadata (`column_name` and `data_type`)

These columns include the column used to calculate each metric and its corresponding Spark data type. The special value of `:table` is used for metrics (such as overall model accuracy) that apply to the entire table, rather than a single column.

Summary statistics are computed for every combination of these grouping columns. [Table 6-1](#) shows the summary statistics present in the profile metrics table.

Table 6-1. Profile metrics table structure (adapted from [Databricks documentation](#))

Column name	Data type	Description
count	bigint	Number of non-null values
num_nulls	bigint	Number of null values in the column
avg	double	Arithmetic mean of the column, ignoring nulls
quantiles	array<double>	Array of 1,000 quantiles
distinct_count	bigint	Approximate number of distinct values in the column
min	double	Minimum value in the column
max	double	Maximum value in the column
stddev	double	Standard deviation of the column
num_zeros	bigint	Number of zeros in the column
num_nan	bigint	Number of not-a-number (NaN) values in the column
min_size	double	Minimum size of arrays or structures in the column
max_size	double	Maximum size of arrays or structures in the column
avg_size	double	Average size of arrays or structures in the column
min_len	double	Minimum length of string and binary values in the column
max_len	double	Maximum length of string and binary values in the column

Column name	Data type	Description
avg_len	double	Average length of string and binary values in the column
frequent_items	array<struct<item: string, count: bigint>>	Top 100 most frequently occurring items
non_null_columns	array<string>	List of columns with at least one non-null value
median	double	Median value of the column
percent_null	double	Percent of null values in the column
percent_zeros	double	Percent of values that are zero in the column
percent_distinct	double	Percent of values that are distinct in the column

In the inference scenario, providing ground truth labels enables the automatic calculation of model performance metrics. [Table 6-2](#) outlines the specific metrics computed for classification problems.

Table 6-2. Metrics computed for classification problems in the profile metrics table (adapted from [Databricks documentation](#))

Column name	Type
accuracy_score	double
confusion_matrix	struct<prediction: string, label: string, count: bigint>
precision	struct<one_vs_all: map<string,double>, macro: double, weighted: double>
recall	struct<one_vs_all: map<string,double>, macro: double, weighted: double>
f1_score	struct<one_vs_all: map<string,double>, macro: double, weighted: double>

[Table 6-3](#) shows the metrics computed for regression problems.

Table 6-3. Metrics computed for regression problems in the profile metrics table (adapted from [Databricks documentation](#))

Column name	Type
<code>mean_squared_error</code>	double
<code>root_mean_squared_error</code>	double
<code>mean_average_error</code>	double
<code>mean_absolute_percentage_error</code>	double
<code>r2_score</code>	double

For classification problems, we can also monitor fairness and bias. First, we must create a boolean slice expression to separate data into two groups. The group where the expression evaluates to True is treated as the *protected group* (the group we are checking for bias against), and the group where the expression evaluates to False is called the *unprotected group*. For example, if the slice is defined by `slice_key = "age < 18"` and `slice_value = True`, we will be checking whether the algorithm is biased against individuals younger than 18 years old.

The monitor then automatically computes fairness metrics by comparing model performance between these groups. These fairness metrics include the following:

predictive_parity

This compares model precision between groups.

predictive_equality

This compares false positive rates.

equal_opportunity

This compares recall.

statistical_parity

This compares the proportions of positive predictions.

At this point, we’ve walked through the profile metrics table, which provides a snapshot of the statistical properties and quality of your data over time. These metrics are useful for understanding how your data behaves

under normal conditions, establishing baselines for monitoring, and quickly detecting unexpected changes such as missing values and anomalies in key features.

Now, let’s take a closer look at the *drift metrics table*, which builds on these profiles to quantify how data distributions change between time windows or against a defined baseline.

Drift Metrics Table

The drift metrics table uses the same grouping columns as the profile metrics table, except that it doesn’t have the `log_type` column. Instead, it includes the following additional columns:

drift_type

This column indicates whether the drift is measured against a fixed `BASELINE` or the `CONSECUTIVE` (previous) time window.

window_cmp

This column specifies the comparison window used when the `drift_type` is `CONSECUTIVE`.

[Table 6-4](#) lists all computed drift metrics and gives descriptions of them.

Table 6-4. Metrics computed for drift detection (adapted from [Databricks documentation](#))

Column name	Data type	Description
count_delta	double	Difference in count
avg_delta	double	Difference in avg
percent_null_delta	double	Difference in percent_null
percent_zeroes_delta	double	Difference in percent_zeros
percent_distinct_delta	double	Difference in percent_distinct
non_null_columns_delta	struct<added: int, missing: int>	Number of columns with any increase or decrease in non-null values
chi_squared_test	struct<statistic: double, pvalue: double>	Chi-square test for drift in distribution, which is calculated for <i>categorical columns</i> only
ks_test	struct<statistic: double, pvalue: double>	Kolmogorov-Smirnov test for drift in distribution, which is calculated for <i>numeric columns</i> only
tv_distance	double	Total variation distance, which is calculated for <i>categorical columns</i> only
l_infinity_distance	double	L-infinity distance, which is calculated for <i>categorical columns</i> only
js_distance	double	Jensen-Shannon distance, which is calculated for <i>categorical columns</i> only
wasserstein_distance	double	Wasserstein distance metric, which is calculated for <i>numeric columns</i> only

Column name	Data type	Description
population_s tability_inde x	double	Population stability index metric, which is calculated for <i>numeric columns</i> only

Let's now go through all the metrics we've mentioned, discuss their meaning, and show you how to calculate them based on practical examples.

Metrics for Monitoring Data Drift

If you're familiar with statistics, you've likely come across methods for comparing two distributions: via hypothesis tests or distance metrics. You may have seen them in statistics courses, ML textbooks, or production monitoring tools used in MLOps.

These same techniques are widely used for *data drift detection*, to help identify shifts between training data and incoming data that could impact model performance. While different resources mention some of these metrics, it's surprisingly hard to find a single resource that brings them all together.

Chi-squared Test

The *chi-squared test* is frequently used to compare the distributions of categorical variables, and it has widespread application in various domains. It's often used in A/B testing (to verify whether users who received a different version of the model react in a different way), and drift detection (to verify that a categorical variable shows the same category frequencies over time).

To illustrate how to calculate the chi-squared statistic and probability value (*p*-value), let's look at the categorical *type_of_meal* variable across the last two monitoring windows.

We start by defining our hypotheses:

Null hypothesis (H_0)

Both datasets come from the same underlying distribution.

Alternative hypothesis (H_1)

The datasets come from different distributions, which suggests that the observed difference is caused by data drift.

Next, we use the data to test the null hypothesis. If we can reject it, we conclude that data drift has occurred. If not, we conclude that the observed differences could be explained by random variation and that there is no evidence of data drift.

Based on the observed values (O) recorded in the profile metrics table, [Table 6-5](#) provides an overview of the category counts for each window.

Table 6-5. Observed meal plan frequencies by window

	Window 1	Window 2	Total
Meal Plan 1	224	629	853
Not Selected	42	141	183
Meal Plan 2	23	65	88
Total	289	835	1124

Now, let's calculate the expected values (E), as shown in [Table 6-6](#).

Table 6-6. Expected meal plan frequencies (E)

	Window 1	Window 2
Meal Plan 1	$289 \times 853 / 1,124 = 219,3$	$835 \times 853 / 1,124 = 633,7$
Not Selected	$289 \times 183 / 1,124 = 47,1$	$835 \times 183 / 1,124 = 136$
Meal Plan 2	$289 \times 88 / 1,124 = 22,6$	$835 \times 88 / 1,124 = 65,3$

The observed chi-squared statistics are calculated as follows:

$$\chi_{obs}^2 = \sum \frac{(E-O)^2}{E} = 0,873$$

This matches the exact value we observed in the drift table for the last period.

Now, let's now calculate the p -value, which is a measure of whether the results of an experiment mean something or are just chance occurrences. It is defined as follows:

$$p = 1 - F_{\chi^2}(\chi_{obs}^2; df)$$

where $F_{\chi^2}(\chi_{obs}^2; df)$ is a cumulative distribution function of the chi-squared distribution.

Also, $df = (r - 1)(c - 1) = 2$, as we have three rows (r) and two columns (c).

Finally, $p = 1 - F_{\chi^2}(0,873; 2) \approx 0,646$.

The *significance level* (α) defines how much evidence we need to reject the null hypothesis. If the *p-value* $\leq \alpha$, we reject H_0 , if the *p-value* $> \alpha$, we fail to reject H_0 .

In our case, the *p-value* is 0,65, which is much greater than a typical α of 0,05. This means the observed difference could easily be due to random chance, so we don't reject the null hypothesis.

Overall, a chi-squared test is a common choice among ML practitioners as it is easy to interpret. It's important to take into consideration that the test is not recommended for categorical features with many low-frequency categories unless the sample size is large.

Kolmogorov-Smirnov Test

The *Kolmogorov-Smirnov test* is a widely used test for investigating the significance of the difference between two distributions, but it's applied to numeric variables. Like the chi-squared test, it evaluates whether two datasets come from the same underlying distribution (H_0) or if a significant shift, which is indicative of data drift, has occurred (H_1).

Let's create a fictional example for the *number_of_week_nights* variable to help us understand how the test is calculated. The first window contains $n = 4$ entries, and the second window contains $m = 6$ entries:

- Window 1: 1, 5, 7, 3
- Window 2: 2, 3, 3, 5, 8, 1

Now we can calculate the cumulative relative frequency (CRF) for both windows and the absolute difference between those values for both windows (see [Table 6-7](#)).

Table 6-7. Calculation of the Kolmogorov-Smirnov (K-S) D-statistic

	Window 1 freq	Window 2 freq	Window 1 CRF	Window 2 CRF	Diff
1	1	1	0,25	0,167	0,083
2		1	0,25	0,333	0,083
3	1	2	0,5	0,667	0,167
5	1	1	0,75	0,833	0,083
7	1		1	0,833	0,167
8		1	1	1	0

The maximum difference is 0,167, which is the value of Kolmogorov-Smirnov statistic $D_{n,m}$.

Rejection of the null hypothesis occurs at a significance level of (α) if

$$D_{n,m} > c(\alpha) \sqrt{\frac{n+m}{n \cdot m}} \text{ (the critical level of D-statistics), where}$$

$$c(\alpha) = \sqrt{-\ln\left(\frac{\alpha}{2}\right) \cdot \frac{1}{2}}.$$

With the significance level 0,05, the critical level of D-statistics is 0,88, $D_{n,m} < D_{crit}$, which means the null hypothesis can't be rejected and there is no data drift.

For simplicity, Databricks also calculates p -value for the Kolmogorov-Smirnov test, which allows it to easily evaluate whether there is data drift.

It's important to note that the Kolmogorov-Smirnov test is sensitive to sample size, so it can lead to false alerts in case of small datasets.

Total Variation Distance

The total variation distance (TVD) is another measure of how two different categorical distributions diverge, and it's frequently used in data drift monitoring. Unlike the chi-squared test, which relies on statistical significance and p -values, the TVD is a *direct numerical measure* of how much two distributions diverge independent of sample size.

Let’s come back to the same example we used to explain the chi-squared test and calculate the TVD. The observed values for the *type_of_meal* variable for each of the windows are presented in [Table 6-8](#).

Table 6-8. Observed frequencies for the `type_of_meal` variable across two time windows

	Window 1	Window 2	Total
Meal Plan 1	224	629	853
Not Selected	42	141	183
Meal Plan 2	23	65	88
Total	289	835	1,124

Now, let’s calculate the relative frequencies for each window and the difference between these relative frequencies for each category in [Table 6-9](#).

Table 6-9. Calculation of absolute differences in relative frequencies between windows

	<i>P</i> (Window 1)	<i>Q</i> (Window 2)	<i>P</i> – <i>Q</i>
Meal Plan 1	224 / 289 = 0,775	629 / 835 = 0,753	0,022
Not Selected	42 / 289 = 0,145	141 / 835 = 0,169	0,024
Meal Plan 2	23 / 289 = 0,080	65 / 835 = 0,078	0,002
Total: 0,048			

We use this formula to calculate the TVD:

$$TVD(P, Q) = \frac{1}{2} \sum_i |P_i - Q_i| = \frac{1}{2} \cdot 0,048 = 0,024$$

This result means that the categorical distribution has changed by only 2,4% between the two windows. Since it’s a small change, we can say the two distributions are very similar, so there is *no meaningful evidence of drift*.

The TVD is easy to compute, and it ranges from 0 (identical distributions) to 1 (completely different distributions). It also does not depend on sample size or statistical thresholds, so it’s intuitive for continuous drift monitoring.

L-Infinity Distance

The *L-infinity distance* (also known as the *maximum absolute difference*) measures the largest change between category frequencies across two distributions.

It is defined as follows:

$$L_{\infty}(P, Q) = \max_i |P_i - Q_i|$$

where P_i and Q_i are the relative frequencies of category i in each window.

Using the same example as in our TVD calculation and our previously computed probabilities, the largest absolute difference between categories is 0,024 (for category “Not Selected”).

This means the maximum shift in any single category between the two time windows is about 2,4%, which confirms that no individual category changed substantially. The distributions remain very similar, and there is no evidence of data drift.

L-infinity distance is easy to compute, ranges from 0 to 1, and captures the biggest individual category change. It’s often used alongside TVD, where TVD shows the overall change and L-infinity highlights the largest category-level difference.

Jensen-Shannon Distance

The Jensen-Shannon distance (JSD) measures how different two probability distributions are from each other. It is a bounded, symmetric, and smoothed version of the Kullback-Leibler divergence D_{kl} , which makes it well-suited for *data drift detection*.

It’s defined as follows:

$$JSD(P \parallel Q) = \sqrt{\frac{1}{2} D_{kl}(P \parallel M) + \frac{1}{2} D_{kl}(Q \parallel M)}$$

where $M = \frac{1}{2}(P + Q)$, and $D_{kl} = \sum_i P_i \log_2 \frac{P_i}{M_i}$.

Using the same previously computed relative frequencies, let’s compute M and KL divergences in [Table 6-10](#).

Table 6-10. Calculation of KL divergence components for Jensen-Shannon distance

	<i>P</i> (Window 1)	<i>Q</i> (Window 2)	<i>M</i>	$D_{kl}(P M)$	$D_{kl}(Q M)$
Meal Plan 1	0,775	0,753	0,764	0,013	0,012
Not Selected	0,145	0,169	0,157	-0,011	-0,012
Meal Plan 2	0,080	0,078	0,079	0,001	0,001
Sum				0,003	0,001

We can now compute the Jensen–Shannon distance:

$$JSD(P \parallel Q) = \sqrt{\frac{1}{2} \cdot (0,003 + 0,001)} = 0,045$$

The value of the JSD runs from 0 for identical distributions to 1 for completely different distributions. In our example, the value of the metric is very small (close to 0), which confirms that the two distributions are almost identical.

While JSD is highly intuitive and stable, it is more computationally expensive than other categorical metrics provided by Databricks.

Wasserstein Distance

The *Wasserstein distance* (also known as the *earth mover’s distance*) measures how much “work” is needed to transform one numeric distribution into another. In one dimension, it equals the area between the two empirical cumulative distribution functions:

$$W_1(P, Q) = \int_{-\infty}^{\infty} |F_p(x) - F_q(x)| dx$$

It’s expressed in the same units as the variable, so it’s easy to interpret.

Let’s use the same example we used to explain the Kolmogorov-Smirnov test. The observations for two windows were as follows:

- Window 1: 1, 5, 7, 3
- Window 2: 2, 3, 3, 5, 8, 1

Using the `number_of_week_nights` variable, let's recalculate the cumulative relative frequencies and the step size Δx for both windows in [Table 6-11](#).

Table 6-11. Calculation of the Wasserstein distance area components

x	Window 1 freq	Window 2 freq	Window 1 CRF	Window 2 CRF	Diff	Δx
1	1	1	0,25	0,167	0,083	1
2		1	0,25	0,333	0,083	1
3	1	2	0,5	0,667	0,167	2
5	1	1	0,75	0,833	0,083	2
7	1		1	0,833	0,167	1
8		1	1	1	0	

The Wasserstein distance is the sum of the areas:

$$W_1(P, Q) = 0,083 \cdot 1 + 0,083 \cdot 1 + 0,167 \cdot 2 + 0,083 \cdot 2 + 0,167 \cdot 1 \approx 0,83$$

It's equal to 0,83 nights, which means that on average, values in one window must shift by $\approx 0,83$ nights to match the other window's distribution. To understand whether this is a lot, we need to compare the shift to the feature's range and standard deviation. Here, the range is 1–8 nights, so 0,83 is $\sim 12\%$ of the range, which can be considered a moderate shift.

One of the advantages of the Wasserstein distance is that it captures *how far* distributions move, not just whether they differ.

Population Stability Index

The Population Stability Index (PSI) measures how much a numeric (or binned) feature's distribution has shifted over time. It compares distributions by aggregating changes across bins. It's easy to interpret and widely used in model monitoring. The PSI is calculated as follows:

$$PSI(P, Q) = \sum_i (Q_i - P_i) \ln\left(\frac{Q_i}{P_i}\right)$$

Let's take the same example again, with the `number_of_week_nights` variable:

- Window 1: 1, 5, 7, 3
- Window 2: 2, 3, 3, 5, 8, 1

Now, we can create bins and compute probabilities in [Table 6-12](#).

Table 6-12. Calculation of the PSI across binned intervals

Interval	P_i (Window 1)	Q_i (Window 2)	$(Q_i - P_i) \ln \left(\frac{Q_i}{P_i} \right)$
$(-\infty, 2.0]$	0,25	0,3333	0,027
$(2.0, 4.0]$	0,25	0,3333	0,027
$(4.0, 6.0]$	0,25	0,1667	0,029
$(6.0, \infty)$	0,25	0,1667	0,029

Finally, we aggregate these individual bin results to calculate the PSI:

$$PSI(P, Q) = 0,027 + 0,027 + 0,029 + 0,029 = 0,112$$

A PSI of 0,112 can be considered a moderate shift in distribution. The following thresholds for PSI (which originated from credit risk monitoring practices) are widely used as industry guidelines:

- $PSI < 0,1$ indicates a negligible shift.
- $0,1 \leq PSI < 0,25$ indicates a moderate shift.
- $PSI \geq 0,25$ indicates a significant shift.

In general, PSI does not have a theoretical cutoff, and its interpretation depends on the variable's natural variability, sample size, and application context. This makes it more difficult to apply PSI consistently in practice, compared to other drift metrics. It is also important to note that PSI is sensitive to the choice and number of bins, which can significantly affect the result.

Now that we have explored the structure of profile and drift metrics tables and how these values are calculated, let's move from theory to implementation. We are going to merge the data from the inference table with the ground truth data and set a monitor on the resulting dataset.

From Inference Tables to Data Quality

Monitoring

In [Chapter 4](#), we enabled an inference table for our custom Model Serving endpoint. This table automatically logs inference requests and responses each time the endpoint is called.

To demonstrate data drift, let's invoke the endpoint using 4,000 records from the `hotel_booking` table: 2,000 selected randomly from the entire dataset and another 2,000 specifically from the corporate market segment. Note that it can take approximately 20 minutes for the inference table to accumulate enough data for analysis:

```
import time

import pandas as pd
import requests
from databricks.sdk import WorkspaceClient
from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig

cfg = ProjectConfig.from_yaml(config_path="../../project_config.yml")
catalog = cfg.catalog
schema = cfg.schema

spark = SparkSession.builder.getOrCreate()
input_data = spark.table(f"{catalog}.{schema}.hotel_booking").toPandas()

# Define required columns for endpoint
required_columns = [
    "Booking_ID",
    "number_of_adults",
    "number_of_children",
    "number_of_weekend_nights",
    "number_of_week_nights",
    "car_parking_space",
    "special_requests",
    "lead_time",
    "type_of_meal",
    "room_type",
    "arrival_month",
    "market_segment_type",
]
```

```

sample_per_phase = 2000

sampled_all = input_data[required_columns].sample(
    n=sample_per_phase, replace=True
)

corporate = input_data[input_data["market_segment_type"] == "Corporate"]
sampled_corporate = corporate[required_columns].sample(
    n=sample_per_phase, replace=True)

sampled_records = pd.concat([sampled_all, sampled_corporate], ignore_index=True)

```

The script for this task, *chapter6_1.call_endpoint.py*, is located in the *notebooks* folder.

We're not changing the approach from [Chapter 4](#), except that we're now including a `client_request_id` field that contains the `Booking_ID`. This addition will be important later, when we join the inference table with the ground truth data.

Here's an example of what the request payload looks like:

```

{
  "client_request_id": "INN36285",
  "dataframe_records": [
    {
      "number_of_adults": 2,
      "number_of_children": 0,
      "number_of_weekend_nights": 0,
      "number_of_week_nights": 1,
      "car_parking_space": 0,
      "special_requests": 0,
      "lead_time": 103,
      "type_of_meal": "Not Selected",
      "room_type": "Room_Type 1",
      "arrival_month": 8,
      "market_segment_type": "Online",
    }
  ]
}

```

Now, let's investigate the structure of an inference table and provide example values.

Inference Table Structure

[Table 6-13](#) shows the structure of the inference table. This table, in its raw form, cannot be used directly for monitoring. All model features are stored in the `request` column as JSON-like strings, rather than as separate columns. Similarly, the response column contains the model predictions in a JSON-like format.

To enable effective monitoring, we first need to process and transform this table into a structured format in which each feature and prediction is represented as an individual column.

Table 6-13. Inference table structure

Column	Data type	Comment	Example
<code>databricks_request_id</code>	string	This is the request ID.	<code>c0e67132-e924-4fb4-b83d-ab5603758baf</code>
<code>request_date</code>	date	This is the UTC date on which the request was received.	<code>2025-10-25</code>
<code>client_request_id</code>	string	This is the user-provided request identifier.	<code>INN34434</code>
<code>request_time</code>	timestamp	This is the timestamp when the request is received.	<code>2025-10-25T21:44:34.637+00:00</code>
<code>status_code</code>	int	This is the HTTP status code.	<code>200</code>
<code>sampling_fraction</code>	double	This is the sampling fraction used in the event that the request is down-sampled.	<code>1.0</code>
<code>execution_duration_ms</code>	bigint	This is the time it took for the model to generate predictions. It does not	<code>5</code>

Column	Data type	Comment	Example
		include overhead network latencies.	
request	string	This is the raw request JSON body that was sent to the Model Serving endpoint.	<pre>{ "client_request_id": "INN34434", "dataframe_records": [{ "number_of_adults": 2, "number_of_children": 0, "number_of_weekend_nights": 0, "number_of_week_nights": 2, "car_parking_space": 0, "special_requests": 0, "lead_time": 317, "type_of_meal": "Meal Plan 1", "room_type": "Room_Type 1", "arrival_month": 7, "market_segment_type": "Offline"}]}</pre>
response	string	This is the raw response JSON body that was returned by the Model Serving endpoint.	<pre>{ "predictions": { "Total price per night": [77.83] } }</pre>
logging_error_codes:	array	These are errors that occurred when the data could not be logged (for example, when request or response exceeded	[]

Column	Data type	Comment	Example
		the maximum allowed size).	
served_entity_id	string	This is the unique ID of the served entity.	40e11be0ddd340eca7e975b13005edc1
requester	string	This is the ID of the user or service principal whose permissions are used to call the serving endpoint. It will be NULL for route-optimized custom model endpoints.	maria@cauchy.io

Now that we're familiar with the structure of the inference table, we can move on to preparing and processing it so we can use it to enable monitoring.

Processing an Inference Table

To process the inference table, we need to specify the structure of the request and the response columns so we can parse it later:

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
```

```

from pyspark.sql.types import (
    ArrayType,
    DoubleType,
    IntegerType,
    StringType,
    StructField,
    StructType,
)

spark = SparkSession.builder.getOrCreate()

request_schema = StructType([
    StructField("dataframe_records", ArrayType(StructType([
        StructField("number_of_adults", IntegerType(), True),
        StructField("number_of_children", IntegerType(), True),
        StructField("number_of_weekend_nights", IntegerType(), True),
        StructField("number_of_week_nights", IntegerType(), True),
        StructField("car_parking_space", IntegerType(), True),
        StructField("special_requests", IntegerType(), True),
        StructField("lead_time", IntegerType(), True),
        StructField("type_of_meal", StringType(), True),
        StructField("room_type", StringType(), True),
        StructField("arrival_month", IntegerType(), True),
        StructField("market_segment_type", StringType(), True),
    ])), True)
])

response_schema = StructType([
    StructField("predictions", StructType([
        StructField("Total price per night",
                    ArrayType(DoubleType()), True)
    ]), True)
])

```

Let's apply the defined schemas and use the `explode` function:

```

from hotel_booking.config import ProjectConfig

cfg = ProjectConfig.from_yaml(
    config_path="../../project_config.yml")
catalog = cfg.catalog
schema = cfg.schema
inf_table = spark.table(
    f"{catalog}.{schema}.hotel_booking_monitoring_payload")

inf_table_parsed = (

```

```

inf_table
.withColumn("p_request",
            F.from_json(F.col("request"),
                        request_schema))
.withColumn("p_response",
            F.from_json(F.col("response"),
                        response_schema))
.withColumn("prediction",
            F.col("p_response.predictions.`Total price per night`")[0])
.withColumn("record",
            F.explode(F.col("p_request.dataframe_records")))
)

```

Next, we select the required columns for the monitoring table and join it with the `hotel_booking` table that contains the ground truth (the actual price). Notice that the final dataframe now contains the request time, the `databricks_request_id` as an identifier, the execution duration, all the model features, the prediction, the model name, and the label:

```

df_final = inf_table_parsed.select(
    "request_time",
    "databricks_request_id",
    "execution_duration_ms",
    F.col("client_request_id").alias("Booking_ID"),
    F.col("record.number_of_adults").alias("number_of_adults"),
    F.col("record.number_of_children").alias("number_of_children"),
    F.col("record.number_of_weekend_nights").alias("number_of_weekend_nights"),
    F.col("record.number_of_week_nights").alias("number_of_week_nights"),
    F.col("record.car_parking_space").alias("car_parking_space"),
    F.col("record.special_requests").alias("special_requests"),
    F.col("record.lead_time").alias("lead_time"),
    F.col("record.type_of_meal").alias("type_of_meal"),
    F.col("record.room_type").alias("room_type"),
    F.col("record.arrival_month").alias("arrival_month"),
    F.col("record.market_segment_type").alias("market_segment_type"),
    "prediction",
    F.lit("hotel-booking-pyfunc").alias("model_name")
)

ground_truth_set = spark.table(f"{catalog}.{schema}.hotel_booking")

df_final_with_status = df_final.join(
    ground_truth_set.select("Booking_ID", "average_price"),
    on="Booking_ID", how="left")

```

Then, we write the results to the Delta table and enable ChangeDataFeed, which is required for incremental updates of the monitor:

```
monitoring_table = f"{catalog}.{schema}.model_monitoring"

df_final_with_status.write.format(
    "delta").mode("append").saveAsTable(monitoring_table)

# Important to update monitoring
spark.sql(f"""ALTER TABLE {monitoring_table}
          SET TBLPROPERTIES (delta.enableChangeDataFeed = true);""")
```

Creating a Data Quality Monitor

Finally, let's create a data quality monitor:

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.dataquality import (
    AggregationGranularity,
    DataProfilingConfig,
    InferenceLogConfig,
    InferenceProblemType,
    Monitor,
)

w = WorkspaceClient()

output_schema = w.schemas.get(full_name=f"{catalog}.{schema}")
monitored_table = w.tables.get(full_name=monitoring_table)

w.data_quality.create_monitor(
    monitor=Monitor(
        object_type="table",
        object_id=monitored_table.table_id,
        data_profiling_config=DataProfilingConfig(
            output_schema_id=output_schema.schema_id,
            assets_dir=(
                f"/Workspace/Shared/DQM/{monitoring_table}"
            ),
            inference_log=InferenceLogConfig(
                problem_type=(
                    InferenceProblemType.INFERENCE_PROBLEM_TYPE_REGRESSION
                ),
                prediction_column="prediction",
```

```

        timestamp_column="request_time",
        granularities=[
            AggregationGranularity.AGGREGATION_GRANULARITY_5_MINUTES
        ],
        model_id_column="model_name",
        label_column="average_price",
    ),
),
),
)

```

We specified the inference log problem type as *regression*, and we provided the prediction column, timestamp column, granularity, model ID column, and label column.

NOTE

In this example, we selected a *five-minute granularity* purely for demonstration purposes. In practice, however, you should choose this parameter carefully based on the specific characteristics of your problem and the expected data dynamics.

The *granularity* defines the time window over which metrics are aggregated and compared for drift detection. If the granularity is set too small, each window may contain few records, and that can cause excessive noise and potentially lead to false drift detections. On the other hand, if the granularity is too large, genuine changes in data distribution or model performance may be smoothed out, and that can make it difficult for you to detect meaningful drift in time. A good approach is to align the granularity with the natural cadence of your data and adjust it empirically, based on the volume and variability of incoming data.

Once you've initiated the monitor creation process, it may take some time before the profile metrics and drift metrics tables are fully populated. This is because Databricks triggers a Lakeflow Spark Declarative Pipeline (SDP) in the background to handle the processing. The duration depends on the size and complexity of the underlying data, as these factors determine the compute time required for the metrics.

At the end of the exercise, under the `hotel_booking` catalog, we'll have the following tables related to monitoring:

```
hotel_booking
├── Tables
│   ├── hotel_booking_monitoring_payload
│   ├── model_monitoring
│   ├── model_monitoring_drift_metrics
│   └── model_monitoring_profile_metrics
```

Using the Dashboard

The dashboard is generated immediately and becomes available in the as-sets directory. The default dashboard contains many visual blocks and can initially feel overwhelming. [Figure 6-3](#) depicts a subset of such a dashboard—notice how the performance drops once the second data sub-set is introduced.

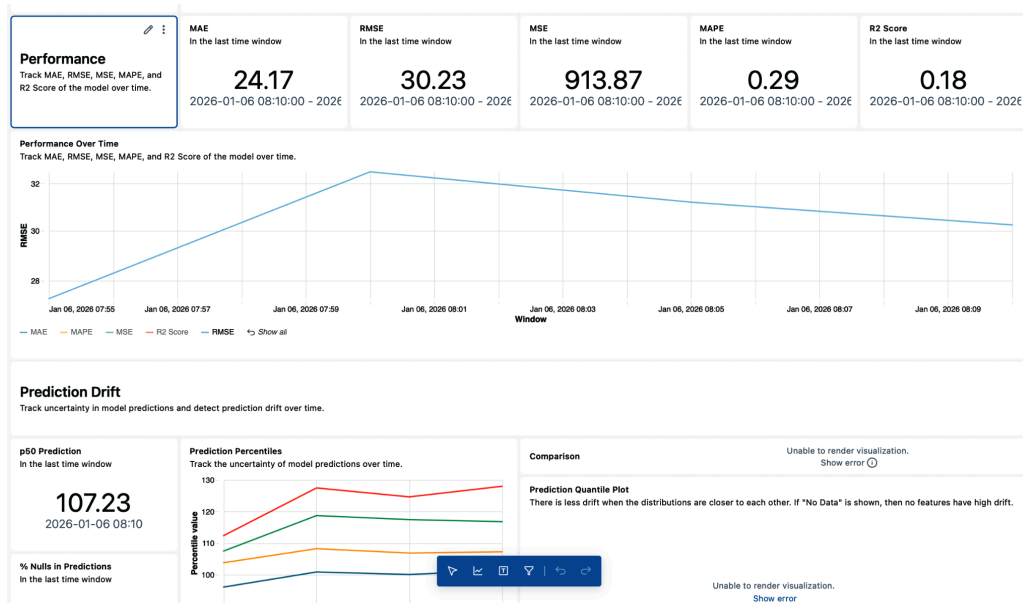


Figure 6-3. A subset of the monitoring dashboard showing performance and prediction drift

Feature drift information is also available in the dashboard. [Figure 6-4](#) demonstrates what categorical features exhibited a drift during the last time window, based on the calculated metrics.

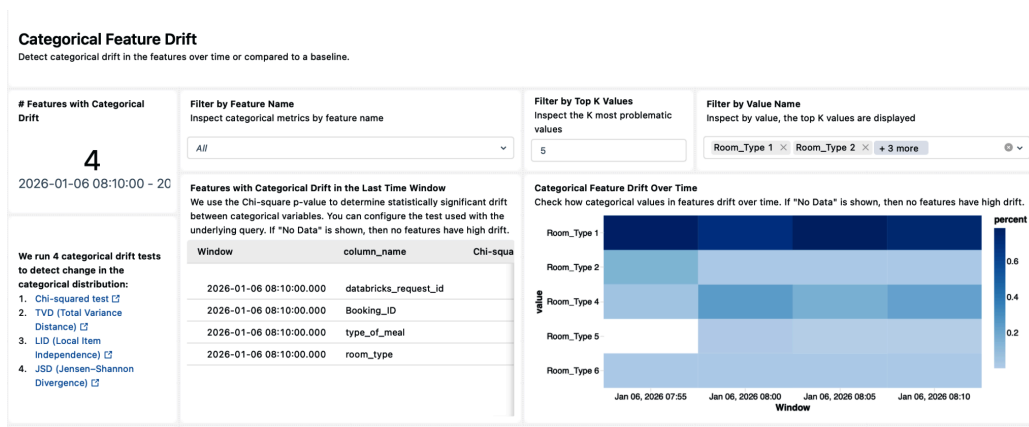


Figure 6-4. Categorical feature drift

You can customize the dashboard to fit your preferences directly through the Databricks UI. Under the hood, Databricks dashboards are defined as JSON files that describe the layout and the underlying SQL queries. It's a good practice to keep these JSON definitions under version control, so you can track changes and collaborate effectively. You can then deploy and manage dashboards programmatically with DABs, to ensure consistency and reproducibility among environments.

Since we expect the constant payload to be generated in the inference table, we need to periodically update the monitoring table and the monitor. Let's discuss the monitor refresh process.

Refreshing the Monitor

Refreshing the monitor is rather simple. We only have to run the `create_refresh()` command, which will trigger the underlying SDP run again:

```
from databricks.sdk.service.dataquality import (
    Refresh,
    RefreshTrigger,
)

w.data_quality.create_refresh(
    object_type="table",
    object_id=monitored_table.table_id,
    refresh=Refresh(
        object_type="table",
        object_id=monitored_table.table_id,
        trigger=RefreshTrigger.MONITOR_REFRESH_TRIGGER_MANUAL,
```

```
),  
)
```

In [Chapter 5](#), we looked at the definition of the monitoring Lakeflow job. Compared to the model training and deployment pipeline, it's rather simple and only includes one task: running `refresh_monitor.py`. We have packaged all the previous logic into this script, which utilizes the `MonitoringManager` class from the `hotel_booking.monitoring` module:

```
from loguru import logger  
from pyspark.sql import SparkSession  
  
from hotel_booking.config import ProjectConfig  
from hotel_booking.monitoring import MonitoringManager  
from hotel_booking.utils.common import create_parser  
  
args = create_parser()  
  
cfg = ProjectConfig.from_yaml(  
    config_path=f"{args.root_path}/files/project_config.yml",  
    env=args.env  
)  
  
spark = SparkSession.builder.getOrCreate()  
  
logger.info("Initializing MonitoringManager")  
manager = MonitoringManager(spark=spark, config=cfg)  
  
logger.info("Starting monitoring table update")  
num_records = manager.update_monitoring_table()  
  
if num_records > 0:  
    logger.info(f"Successfully processed {num_records} records")  
    manager.refresh_monitor()  
    logger.info("Monitor refresh completed")  
else:  
    logger.info("No new records to process")
```

The task has the following steps:

1. Check whether new data is populated in the inference table.

2. If new data is available, parse it, join it with the ground truth, and update the monitoring table.
3. Once the monitoring table is updated, refresh the monitor. This action will update the profile and drift metrics tables, plus the corresponding dashboard.

The ML pipeline we defined earlier is set to run weekly, while the monitoring pipeline runs daily. We discussed the cadence of model retraining in [Chapter 5](#).

The frequency we use is illustrative. Your monitoring frequency should align with how quickly you need to detect and respond to issues. Daily monitoring works well for batch inference systems where predictions are consumed in aggregate. But if your model serves real-time requests and degradation directly impacts users, you may want to monitor hourly or even continuously. You should also consider the cost implications: higher monitoring frequency means higher costs. The goal is to find the sweet spot that keeps you informed of the model quality without needlessly burning through your budget.

Having a monitoring dashboard and keeping the underlying tables updated is a good start, but it's not enough on its own. No one is going to sit there staring at metrics, waiting for something to break. When things go wrong, you want to find out immediately. Setting up alerting is one way of dealing with it.

Alerting

There are different ways to set up alerting, and the most convenient one is to use DABs (which we covered in [Chapter 5](#)). The alert deployment is specified in the *resources/alert.yml* file.

To create an alert, we first need to create a SQL warehouse, which we will use to run the specified query according to the predefined schedule. In our example, we choose the smallest serverless SQL warehouse with auto termination of five minutes to reduce the costs. At the moment of writing this book, it's not possible to specify a usage policy for a SQL warehouse, so we've provided tags for cost tracking purposes:

```
resources:
  sql_warehouses:
    hotel_booking_sql_warehouse:
      name: hotel_booking_sql_warehouse
      cluster_size: 2X-Small
      enable_serverless_compute: true
      max_num_clusters: 1
      auto_stop_mins: 5
      warehouse_type: PRO
      tags:
        custom_tags:
          - key: "project"
            value: "hotel_booking"
```

Now, let's define an alert. This one triggers when the percentage of records with a mean absolute error above 30 in the `model_monitoring_profile_metrics` table exceeds 45%. Note that we're pointing at the table in the production catalog. The query runs daily at 10:00 Amsterdam time, using the warehouse we configured earlier, and when triggered, it notifies users via email or a configured notification destination:

```
alerts:
  my_alert:
    display_name: hotel-booking-mae-alert
    evaluation:
      notification:
        subscriptions:
          - user_email: "maria@cauchy.io"
            # - destination_id: "12345"
        comparison_operator: GREATER_THAN
      source:
        name: 'percentage_higher_than_30'
      threshold:
        value:
          double_value: 45
    query_text: SELECT (COUNT(CASE WHEN mean_absolute_error > 30 THEN 1 END)
* 100.0 / COUNT(CASE WHEN mean_absolute_error IS NOT NULL AND NOT
  isnan(mean_absolute_error) THEN 1 END)) AS percentage_higher_than_30
  FROM mlops_prd.hotel_booking.model_monitoring_profile_metrics
    schedule:
      quartz_cron_schedule: "0 0 10 * * ?"
      timezone_id: Europe/Amsterdam
```

```
pause_status: ${var.schedule_pause_status}
warehouse_id: ${resources.sql_warehouses.hotel_booking_sql_warehouse.id}
```

Note that the alert won't run in acceptance because we set `schedule_pause_status` to `PAUSED` for that target, and it won't run in development because that target has development mode enabled. We covered this when we went through the `databricks.yml` file definition in [Chapter 5](#).

It's possible to configure different notification destinations. [Figure 6-5](#) shows how workspace admins can configure notification destinations. Some destinations like Microsoft Teams, PagerDuty, and Slack are supported out-of-the-box, but a generic webhook option is also supported.

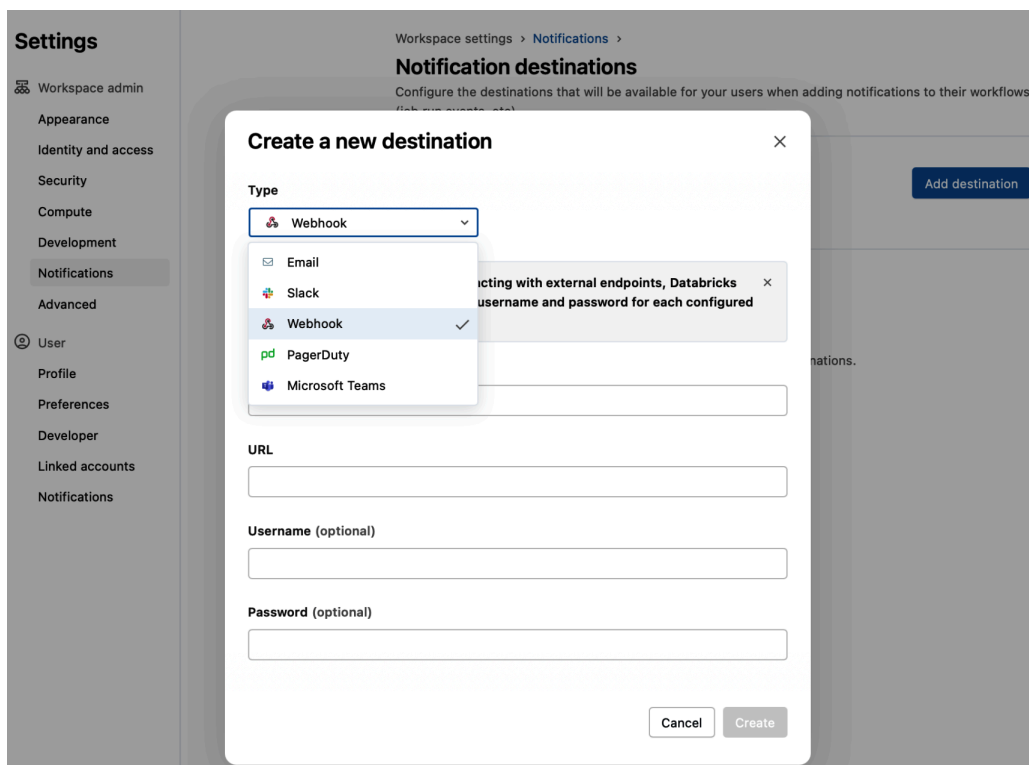


Figure 6-5. Configuring notification destinations in Databricks workspace

So far, we've talked about monitoring that's related to model features and model performance, but as we discussed at the beginning of the chapter, there are more things to monitor. We'll come back to monitoring in [Chapter 9](#), where we'll discuss the cost dimension. Cost matters for any production system, but it is especially important for LLMs; token-based pricing means spend can grow quickly and often invisibly, which makes it one of the first operational signals you'll want to track. Because we apply two different usage policies across the book, by the end of the MLOps and LLMOps chapters, you will be able to see cost segregated between them.

Conclusion

In this chapter, we explored the many facets of monitoring ML systems in production. If there's one takeaway from this chapter, it's that monitoring is hard. Deciding what to measure, setting thresholds that catch real problems, routing alerts to the right people, and actually acting on what you find—all of it takes iteration. You won't get it right on the first try, and that's fine. The important thing is having the infrastructure in place so you can observe, learn, and adjust.

A model in production without monitoring is a liability. You may get lucky, and it may keep performing well. Or it may silently degrade for weeks while your users make decisions based on increasingly unreliable predictions. Monitoring doesn't guarantee your model will work, but at least you'll know when it isn't working.

This chapter closes the MLOps part of the book. The next three chapters turn to LLM-based systems, and you'll find that the operational concerns don't disappear so much as change shape. Monitoring is one of those threads: it returns for agents in [Chapter 9](#), where nondeterministic, open-ended output makes “Is it still working?” a harder question to answer for LLMs than it was for a model with a fixed output schema.

- 1** Read [the full article](#).
- 2** Check out Santiago Viquez's article [“Don't Let Yourself Be Fooled by Data Drift”](#). It discusses this problem in detail.

Previous chapter

Next chapter

< [5. Machine Learning Model
Deployment](#)

[7. Foundation Models and Context
Engineering](#) >